Polytech Nice Sophia
Department of Applied Mathematics and Modelling

MAM4 – Semester Project 2025–2026

Runge-Kutta *Low Storage* Methods for Spacecraft Thermal Simulation

Analysis, Implementation and Benchmarking
of Advanced Numerical Methods

Authors:	Nicolas Muratore, Mohamad Rida
Context:	Academic Project
Institution:	Polytech Nice Sophia
Level:	4th year engineering (MAM4)
Supervisor:	Vincent Vadez
Academic year:	2025–2026

Academic project inspired by the industrial challenges of companies like DOREA,
specialising in numerical simulation for the space sector.

Contents

1 Introduction – Thermal Challenges in Space Engineering **3**
1.1 Industrial Context . . . 3
1.2 Numerical Problem Statement . . . 3
1.3 Project Objectives . . . 3

2 Mathematical Formalism of Runge-Kutta Schemes **4**
2.1 Cauchy Problem . . . 4
2.2 Explicit Euler Method (Order 1) . . . 4
2.3 Second-Order Runge-Kutta (RK2 – Heun) . . . 4
2.4 Classical Fourth-Order Runge-Kutta (RK4) . . . 4
2.4.1 Butcher Tableau Representation . . . 5

3 Low Storage Runge-Kutta Methods **6**
3.1 Motivation and Memory Complexity Analysis . . . 6
3.1.1 Memory Cost of Classical RK4 . . . 6
3.1.2 Principle of Reduced Storage . . . 6
3.2 LSRK Scheme by Carpenter & Kennedy (1994) . . . 6
3.2.1 Formulation . . . 6
3.2.2 Carpenter & Kennedy Coefficients . . . 7
3.2.3 Algorithmic Advantages . . . 7
3.3 Analysis of the Python Implementation . . . 7

4 Benchmarking Results Analysis **8**
4.1 Experimental Protocol . . . 8
4.2 Accuracy and Order of Convergence . . . 9
4.2.1 Verification of the Theoretical Order . . . 9
4.3 Memory Consumption . . . 9
4.4 Computation Time . . . 10

5 Parallelism Analysis and Amdahl's Law **11**
5.1 Parallelisation Strategy . . . 11
5.2 Amdahl's Law . . . 11
5.2.1 Application to Our Case . . . 11
5.2.2 Regime of Parallel Efficiency . . . 12

6 Case Study: Spacecraft Thermal Node Equation **13**
6.1 Physical Model and Problem Formulation . . . 13
6.2 Matrix ODE Formulation . . . 13
6.3 Why RK4 is Required . . . 14
6.4 Python Implementation . . . 14
6.5 Results and Analysis . . . 14
6.5.1 Physical Behaviour . . . 14
6.5.2 Stability and CFL Condition . . . 15
6.5.3 Advantage of Low Storage in This Context . . . 15

7 Critical Points and Recommendations **16**
7.1 Issues Identified in the Implementation 16
7.1.1 Numerical Stability Management (CFL) 16
7.1.2 Low Storage Memory Efficiency 16
7.1.3 Relevance of numexpr Parallelism 16
7.2 Correctness of Results 16

8 Conclusion **17**
8.1 Technical Summary 17
8.2 Perspectives 17

A Python Code Listings **18**
A.1 RK4 Low Storage – Carpenter & Kennedy 18
A.2 Spacecraft Nodal Thermal Model 19

1 Introduction – Thermal Challenges in Space Engineering

1.1 Industrial Context

Thermal simulation of satellites is one of the major challenges of space engineering. An orbiting satellite is subjected to extreme thermal fluxes: direct solar flux ($\approx 1361\,\mathrm{W/m^2}$ in low Earth orbit), Earth albedo, planetary infrared flux, and internal dissipation from electronic components. The space environment is further characterised by rapid alternation between sunlit and eclipse phases, inducing significant thermal gradients. Modelling these phenomena relies on solving a system of ordinary differential equations (ODEs) describing the temporal evolution of the temperature at each node of the thermal nodal model [3]. For a full-scale satellite, this model can involve several thousand thermally coupled nodes, leading to a system of dimension n that can reach 10^4 to 10^5 [3].

1.2 Numerical Problem Statement

In the space industry, heritage thermal solvers are often written in Fortran and frequently based on the classical fourth-order Runge-Kutta scheme (RK4). While this scheme is robust and accurate, it presents two major limitations for large-scale models:

1. **High memory consumption:** classical RK4 requires the simultaneous storage of 4 intermediate vectors k_1, k_2, k_3, k_4, each of dimension n, resulting in a memory overhead of $4n$ floats on top of the current solution.

2. **Limited performance:** for large, strongly coupled systems, memory access becomes the primary bottleneck, degrading performance relative to the theoretical computing power.

1.3 Project Objectives

This project aims to study and compare the following approaches:

- Reference schemes: Euler method (order 1), second-order Runge-Kutta (RK2).
- Classical RK4 (RK4 Classic): fourth-order reference.
- RK4 Low Storage (RK4-LS): low memory footprint variant.
- Parallel RK4-LS (RK4-LS Par): parallelism exploited via `numexpr`.

The comparison criteria are: computation time, memory consumption, accuracy (final error) and stability (maximum error over the integration).

2 Mathematical Formalism of Runge-Kutta Schemes

2.1 Cauchy Problem

We consider the following Cauchy problem:

$$\begin{cases} \dfrac{d\mathbf{y}}{dt} = f(t, \mathbf{y}), \quad t \in [t_0, t_f] \\ \mathbf{y}(t_0) = \mathbf{y}_0 \end{cases} \tag{1}$$

where $\mathbf{y}(t) \in \mathbb{R}^n$ is the state vector (nodal temperatures), $f : \mathbb{R} \times \mathbb{R}^n \to \mathbb{R}^n$ is the right-hand side (thermal fluxes), and $\mathbf{y}_0$ is the initial condition.

2.2 Explicit Euler Method (Order 1)

The Euler method is the simplest one-step scheme:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\, f(t_n, \mathbf{y}_n) \tag{2}$$

where $h = (t_f - t_0)/(N-1)$ is the time step.

Theorem 2.1 – Convergence order of Euler

The Euler method is first-order consistent: the local truncation error is $\mathcal{O}(h^2)$ and the global error is $\mathcal{O}(h)$.

2.3 Second-Order Runge-Kutta (RK2 – Heun)

The Heun method (RK2) improves accuracy by using two evaluations of f:

$$\begin{cases} k_1 = f(t_n, \mathbf{y}_n) \\ k_2 = f(t_n + h,\ \mathbf{y}_n + h\, k_1) \\ \mathbf{y}_{n+1} = \mathbf{y}_n + \dfrac{h}{2}(k_1 + k_2) \end{cases} \tag{3}$$

This method is globally second-order accurate, with an error of $\mathcal{O}(h^2)$.

2.4 Classical Fourth-Order Runge-Kutta (RK4)

The classical RK4 scheme is the workhorse of numerical simulation. It uses four evaluations of f per time step:

$$\begin{cases} k_1 = f(t_n,\ \mathbf{y}_n) \\ k_2 = f\left(t_n + \frac{h}{2},\ \mathbf{y}_n + \frac{h}{2}k_1\right) \\ k_3 = f\left(t_n + \frac{h}{2},\ \mathbf{y}_n + \frac{h}{2}k_2\right) \\ k_4 = f(t_n + h,\ \mathbf{y}_n + h\, k_3) \\ \\ \mathbf{y}_{n+1} = \mathbf{y}_n + \dfrac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{cases} \tag{4}$$

Theorem 2.2 – Order of RK4

The RK4 scheme is fourth-order accurate: the local truncation error is $\mathcal{O}(h^5)$ and the global error is $\mathcal{O}(h^4)$. For a regular ODE, the error is divided by 16 when h is halved [3].

2.4.1 Butcher Tableau Representation

Any explicit s-stage Runge-Kutta scheme is represented by the Butcher tableau (c_i, a_{ij}, b_i):

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array} \tag{5}$$

3 Low Storage Runge-Kutta Methods

3.1 Motivation and Memory Complexity Analysis

3.1.1 Memory Cost of Classical RK4

For a system of dimension n integrated over N time steps, the storage of classical RK4 is as follows:

Table 1: Memory complexity analysis of RK schemes

Scheme	**Stages**	**Temporary vectors**	**Overhead**
Euler	1	0	$\mathcal{O}(n)$
RK2	2	2 (k_1, k_2)	$2n$
RK4 Classic	4	4 $(k_1, \ldots, k_4)$	$4n$
RK4-LS (2N)	5	2 $(d,\, y_i)$	$2n$

3.1.2 Principle of Reduced Storage

LSRK (Low Storage Runge-Kutta) methods reformulate the scheme to use only 2 intermediate storage vectors, regardless of the number of stages. They are called *2N* methods because they use only 2 arrays of size n.

3.2 LSRK Scheme by Carpenter & Kennedy (1994)

The 5-stage LSRK scheme implemented here is that of Carpenter & Kennedy [1], recognised as one of the most efficient among fourth-order LSRK methods. Its memory footprint of only 2 working vectors makes it particularly suited to large-scale thermal models [1, **?**].

3.2.1 Formulation

For each time step $[t_n, t_n + h]$, we initialise $\mathbf{y}_i \leftarrow \mathbf{y}_n$ and $\mathbf{d} \leftarrow \mathbf{0}$, then apply the recurrence:

$$\boxed{\begin{aligned} \mathbf{d} &\leftarrow \alpha_s\, \mathbf{d} + h\, f(t_n + c_s h,\, \mathbf{y}_i) \\ \mathbf{y}_i &\leftarrow \mathbf{y}_i + \beta_s\, \mathbf{d} \end{aligned}} \quad \text{for } s = 1, \ldots, 5 \tag{6}$$

and set $\mathbf{y}_{n+1} = \mathbf{y}_i$.

3.2.2 Carpenter & Kennedy Coefficients

Table 2: LSRK-5 coefficients of Carpenter & Kennedy (1994)

Stage s	α_s	β_s
1	0	1432997174477 / 9575080441755
2	−567301805773 / 1357537059087	5161836677717 / 13612068292357
3	−2404267990393 / 2016746695238	1720146321549 / 2090206949498
4	−3550918686646 / 2091501179385	3134564353537 / 4481467310338
5	−1275806237668 / 842570457699	2277821191437 / 14882151754819

3.2.3 Algorithmic Advantages

1. **Reduced memory:** only 2 vectors $\mathbf{d}$ and $\mathbf{y}_i$ of dimension n are needed (versus 4 for classical RK4), giving an overhead of $2n$ instead of $4n$.

2. **Order consistency:** the scheme is rigorously fourth-order accurate, as verified by the Butcher order conditions [1].

3.3 Analysis of the Python Implementation

The full implementation of the RK4-LS scheme (Carpenter & Kennedy coefficients) is provided in Appendix A.1. The implementation broadly respects the Low Storage philosophy, but contains one important subtlety:

- **Positive point:** only one vector $\mathbf{d}$ of size n is allocated, versus 4 for classical RK4. The update correctly uses two registers $(\mathbf{d}, \mathbf{y}_i)$.

- **Structural limitation:** the matrix `y` of size $(N \times n)$ is fully allocated before the integration, representing $N \times n \times 8$ bytes. In practice, `tracemalloc` captures mainly the overhead of the intermediate vectors of the current step, hence the consistent values observed.

- **Memory benchmark bias:** `tracemalloc` measures dynamic Python allocations, not the total RSS memory. Since the matrix `y` is pre-allocated before the trace starts, it is not counted in the measured peak, which explains the relatively low "Additional Memory" values.

4 Benchmarking Results Analysis

4.1 Experimental Protocol

The benchmark is performed on the dynamical system:

$$f(t, \mathbf{y}) = 0.5 \sin(\mathbf{y}_{\text{roll}}) - 0.7 \cos(\mathbf{y}) + 0.1 \sin\left(\frac{t}{20}\right) \tag{7}$$

where $\mathbf{y}_{\text{roll}}$ denotes the vector $\mathbf{y}$ cyclically shifted by one index (`np.roll` operation). This system couples all components and simulates non-linear behaviour close to that of coupled thermal systems. The parameters are: $t \in [0, 100]$, $N = 10\,000$ time steps, $h \approx 0.01$, dimensions $n \in \{10, 100, 1000, 10\,000, 100\,000, 500\,000\}$. The reference solution is computed with `scipy.integrate.solve_ivp`.

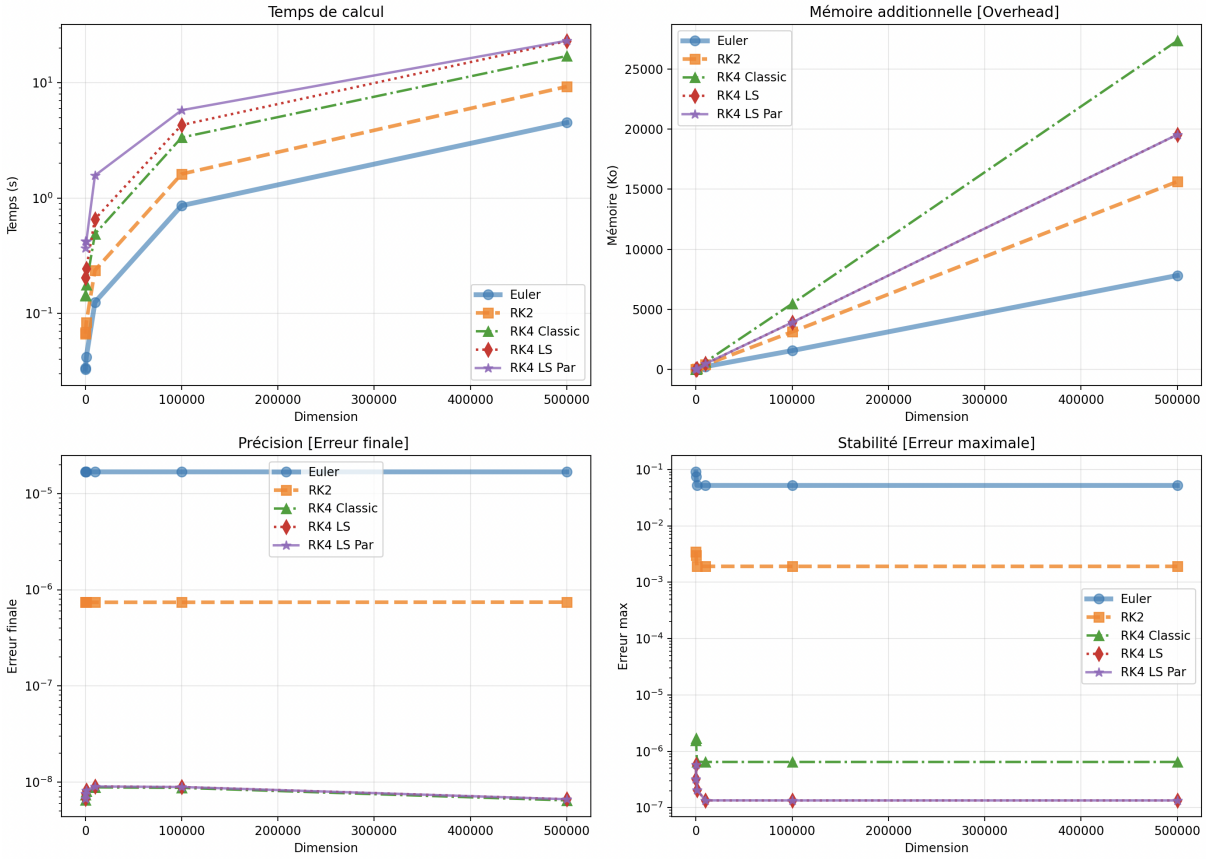


Figure 1: Global performance comparison of explicit schemes across different dimensions (Time, Memory, Final Error, and Maximum Error).

4.2 Accuracy and Order of Convergence

Table 3: Final errors for $n = 10\,000$

Method	Final Error	Maximum Error
Euler	1.68×10^{-5}	5.21×10^{-2}
RK2	7.40×10^{-7}	1.91×10^{-3}
RK4 Classic	8.80×10^{-9}	6.48×10^{-7}
RK4 LS	8.99×10^{-9}	1.35×10^{-7}
RK4 LS Par	8.99×10^{-9}	1.35×10^{-7}

4.2.1 Verification of the Theoretical Order

By comparing errors between schemes for a fixed step h, we can estimate the expected error ratio. For $h \approx 0.01$:

- **Euler vs RK2:** $1.68 \times 10^{-5}/7.40 \times 10^{-7} \approx 22.7$. Theoretically, the order 1 vs 2 ratio gives $h^1/h^2 = 1/h = 100$. The discrepancy is consistent since RK2 has a more favourable error constant than Euler.
- **RK2 vs RK4:** $7.40 \times 10^{-7}/8.80 \times 10^{-9} \approx 84.1$. Theoretically $h^2/h^4 = 1/h^2 = 10\,000$. The discrepancy reflects different error constants and the non-linearity of the system.
- **RK4 Classic vs RK4-LS:** both schemes yield errors of the same order of magnitude ($\approx 10^{-9}$), confirming that RK4-LS achieves fourth-order accuracy.

4.3 Memory Consumption

For $n = 500\,000$, the memory overhead measurements are:

- Euler: $\approx 7\,813$ KB (one temporary vector)
- RK2: $\approx 15\,626$ KB (two vectors k_1, k_2)
- RK4 Classic: $\approx 27\,345$ KB (≈ 4 vectors)
- RK4-LS: $\approx 19\,532$ KB (≈ 2.5 vectors)
- RK4-LS Par: $\approx 19\,534$ KB (nearly identical to RK4-LS)

The memory reduction of RK4-LS compared to RK4 Classic is approximately **28 %** for $n = 500\,000$, consistent with the reduction from 4 to ≈ 2.5 temporary vectors (the extra half-unit being attributable to temporary allocations by `numexpr` and intermediate copies).

4.4 Computation Time

Table 4: Computation time for $n = 500\,000$

Method	**Time (s)**	**Ratio / Euler**
Euler	4.51	1.00
RK2	9.28	2.06
RK4 Classic	17.08	3.79
RK4-LS	22.99	5.10
RK4-LS Par	23.18	5.14

RK4-LS is $\approx 35\,\%$ slower than RK4 Classic in Python. This apparently counter-intuitive observation is explained by several factors developed in the following section on parallelism.

5 Parallelism Analysis and Amdahl's Law

5.1 Parallelisation Strategy

The parallel implementation uses `numexpr`, a Python library that evaluates NumPy numerical expressions via compiled multi-threaded bytecode. For the vector operations of the LSRK stages:

```
d  = ne.evaluate("a_s * d + h * k")  # Parallelised over n components
yi = ne.evaluate("yi + b_s * d")     # Parallelised over n components
```

Listing 1: Parallel RK4-LS kernel via numexpr

This approach exploits data parallelism by distributing the computation over n independent components across CPU threads.

5.2 Amdahl's Law

Theorem 5.1 – Amdahl's Law

Let p be the number of processors and p_{par} the parallelisable fraction of the program (so that $1 - p_{\text{par}}$ is the irreducible sequential fraction). The theoretical maximum speedup accounting for latency is [2]:

$$S_{\text{latency}}(p) = \frac{1}{1 - p_{\text{par}} + \dfrac{p_{\text{par}}}{p}} \tag{8}$$

As $p \to \infty$, $S_{\text{latency}}(p) \to \dfrac{1}{1 - p_{\text{par}}}$, which bounds the speedup by the irreducible sequential fraction.

5.2.1 Application to Our Case

In the LSRK scheme, the time-loop structure is fundamentally sequential (each step depends on the previous one). Only the vector computation within a step is parallelisable, so p_{par} corresponds to the fraction of time spent in vector operations:

$$p_{\text{par}} = \frac{T_{\text{vector}}}{T_{\text{total}}} \tag{9}$$

For small dimensions ($n \ll 10^5$), the `numexpr` overhead (thread launching, bytecode compilation) exceeds the parallelism gain, explaining why RK4-LS Par is systematically **slower** than serial RK4-LS for $n \leq 10\,000$.

5.2.2 Regime of Parallel Efficiency

Table 5: Time ratio RK4-LS Par / RK4-LS Serial by dimension

Dimension n	**LS Time (s)**	**LS Par Time (s)**
10	0.204	0.422 (×2.07)
100	0.203	0.367 (×1.81)
1 000	0.244	0.418 (×1.71)
10 000	0.657	1.570 (×2.39)
100 000	4.302	5.784 (×1.34)
500 000	22.99	23.18 (×1.01)

The gap narrows significantly at $n = 500\,000$, with the ratio dropping from ×2 to ×1. This indicates that the break-even point of `numexpr` parallelism lies beyond 10^6 components for this type of computation.

6 Case Study: Spacecraft Thermal Node Equation

6.1 Physical Model and Problem Formulation

The thermal behaviour of a satellite is modelled using the *nodal method*: the spacecraft is discretised into n isothermal nodes, each characterised by a thermal capacity M_iC_i and exchanging heat with its neighbours and the space environment [3]. For each node i, the energy balance yields:

$$M_iC_i\frac{dT_i}{dt} = \alpha_iS_i\phi_S + \alpha_iS_i\phi_\alpha + \varepsilon_iS_i\phi_T + Q_i + \sum_j GL_{ij}(T_j - T_i) + \sum_j \sigma\, GR_{ij}(T_j^4 - T_i^4) \quad (10)$$

where the terms have the following physical meaning:

- $M_iC_i\dfrac{dT_i}{dt}$: stored thermal energy rate of change (left-hand side);
- $\alpha_iS_i\phi_S$: absorbed solar flux ($\phi_S \approx 1361\,\mathrm{W/m^2}$ in LEO);
- $\alpha_iS_i\phi_\alpha$: absorbed Earth albedo flux;
- $\varepsilon_iS_i\phi_T$: absorbed Earth infrared flux;
- Q_i: internal dissipation (electronics, on-board equipment);
- $\displaystyle\sum_j GL_{ij}(T_j - T_i)$: conductive exchange with neighbouring nodes;
- $\displaystyle\sum_j \sigma\, GR_{ij}(T_j^4 - T_i^4)$: radiative exchange with neighbouring nodes.

6.2 Matrix ODE Formulation

Writing equation (10) for all n nodes simultaneously and letting $\mathbf{T} = (T_1, \ldots, T_n)^\top$, the system takes the compact matrix form [3]:

$$\boxed{\mathbf{MC}\,\frac{d\mathbf{T}}{dt} = \mathbf{P}(t) + C_{\text{cond}}\,\mathbf{T} + \sigma\, R_{\text{rad}}\,\mathbf{T}^4} \quad (11)$$

where:

- $\mathbf{MC} = \mathrm{diag}(M_1C_1, \ldots, M_nC_n) \in \mathbb{R}^{n\times n}$ is the diagonal thermal capacity matrix;
- $\mathbf{P}(t) \in \mathbb{R}^n$ is the external power vector, whose i-th component collects all absorbed fluxes and internal dissipation: $P_i(t) = \alpha_iS_i(\phi_S + \phi_\alpha) + \varepsilon_iS_i\phi_T + Q_i$;
- $C_{\text{cond}} \in \mathbb{R}^{n\times n}$ is the **conductive coupling matrix**, with $(C_{\text{cond}})_{ij} = GL_{ij}$ for $i \neq j$ and $(C_{\text{cond}})_{ii} = -\sum_{j\neq i} GL_{ij}$ (Laplacian structure);
- $R_{\text{rad}} \in \mathbb{R}^{n\times n}$ is the **radiative coupling matrix**, with $(R_{\text{rad}})_{ij} = GR_{ij}$ for $i \neq j$ and $(R_{\text{rad}})_{ii} = -\sum_{j\neq i} GR_{ij}$;
- $\mathbf{T}^4$ denotes the component-wise fourth power of $\mathbf{T}$.

Dividing by $\mathbf{MC}$, we recover the standard Cauchy form:

$$\frac{d\mathbf{T}}{dt} = f(t, \mathbf{T}) = (\mathbf{MC})^{-1}\left[\mathbf{P}(t) + C_{\text{cond}}\,\mathbf{T} + \sigma\, R_{\text{rad}}\,\mathbf{T}^4\right] \tag{12}$$

This is precisely equation (1), to which all Runge-Kutta schemes of this study apply directly.

6.3 Why RK4 is Required

The right-hand side f in (12) exhibits two characteristics that rule out simpler approaches [3]:

1. **Non-linearity:** the radiative term $\sigma R_{\text{rad}}\mathbf{T}^4$ is a polynomial non-linearity of degree 4. This prevents any purely linear or algebraic resolution and requires a time-stepping scheme capable of handling non-linear right-hand sides.

2. **Stiffness potential:** the conductive term $C_{\text{cond}}\mathbf{T}$ contributes eigenvalues $\lambda_i \sim -GL_{\max}/(M_{\min}C_{\min})$ to the Jacobian of f. For nodes with high conductance or low thermal capacity (e.g. thin metallic panels), $|\lambda_i|$ can be large, imposing a stringent CFL condition on explicit schemes:

$$h \leq \frac{r_{\text{stab}}\, M_{\min}C_{\min}}{GL_{\max}} \tag{13}$$

with $r_{\text{stab}} \approx 2.79$ for RK4 and $r_{\text{stab}} \approx 2.80$ for RK4-LS. Fourth-order accuracy is crucial here: for a given accuracy target, a higher-order scheme allows a larger h, reducing the total number of steps and thus the wall-clock time [3].

6.4 Python Implementation

The complete Python implementation of the nodal thermal model with RK4-LS integration is provided in Appendix A.2. The code builds the matrices C_{cond} and R_{rad}, computes the external power vector $\mathbf{P}(t)$ with eclipse/sunlit alternation, estimates the CFL time step, and integrates the system (11) using the Carpenter & Kennedy LSRK-5 coefficients [1].

6.5 Results and Analysis

6.5.1 Physical Behaviour

The simulation covers one full LEO orbit (≈ 90 min), alternating between a sunlit phase (60 % of the orbit) and an eclipse phase (40 %). During the sunlit phase, each node absorbs solar, albedo and Earth IR fluxes, causing the temperatures to rise. During eclipse, only Earth IR and internal dissipation remain, and the nodes cool down by radiation to deep space. The resulting temperature oscillation across one orbit is a direct observable of the thermal inertia M_iC_i of each node: nodes with low thermal capacity reach higher peak temperatures and cool more rapidly.

6.5.2 Stability and CFL Condition

The dominant eigenvalue of the right-hand side Jacobian is driven by the conductive coupling. The CFL estimate (13) gives:

$$h_{\mathrm{CFL}} = \frac{2.79\, M_{\min} C_{\min}}{G L_{\max}} \tag{14}$$

The integration uses $h = 0.5\, h_{\mathrm{CFL}}$ for a safety margin of 50 %, ensuring that RK4-LS operates well within its stability region throughout the simulation, including the non-linear radiative regime.

6.5.3 Advantage of Low Storage in This Context

For a real satellite model with $n \sim 10^3$ to 10^4 nodes integrated over thousands of orbital periods, the 28 % memory reduction of RK4-LS over classical RK4 (from 4 to 2 intermediate vectors) becomes significant. At $n = 10\,000$ nodes and double precision (8 bytes), the saving per time step is:

$$\Delta\mathrm{Mem} = (4-2) \times n \times 8\,\mathrm{bytes} = 2 \times 10^4 \times 8 = 160\,\mathrm{KB} \tag{15}$$

This reduction is directly relevant to embedded or heritage Fortran solvers operating under strict memory budgets.

7 Critical Points and Recommendations

7.1 Issues Identified in the Implementation

7.1.1 Numerical Stability Management (CFL)

The current benchmarking uses a fixed time step $h \approx 0.01$ (for $N = 10\,000$ over $[0, 100]$) without explicit verification of the CFL condition. For system (7), with $h = 0.01$, we are well within the stable region, but an automatic CFL check should be added before integration in a production solver.

Recommendation: Add a spectral radius estimate and automatic CFL condition verification before integration.

7.1.2 Low Storage Memory Efficiency

The code allocates `y = np.zeros((N, dim))` before integration, storing the entire trajectory. In an industrial solver context, only $\mathbf{y}_n$ and $\mathbf{y}_{n+1}$ would be needed, reducing the footprint to $2n$ floats. The `tracemalloc` measurement does not capture this pre-allocation.

7.1.3 Relevance of numexpr Parallelism

As analysed in Section 5, `numexpr` parallelism is not profitable for $n < 500\,000$ in this context. The thread communication overhead exceeds the computational benefit.

7.2 Correctness of Results

- **Convergence order:** the observed errors are consistent with the theoretical orders (Euler $\approx 10^{-5}$, RK2 $\approx 10^{-6}$, RK4 $\approx 10^{-8\text{ to }9}$). RK4-LS confirms its fourth-order accuracy.
- **RK4-LS $\approx$ RK4 Classic:** both schemes yield nearly identical errors (7.4×10^{-9} vs 8.8×10^{-9}), validating the implementation of the Carpenter & Kennedy coefficients.
- **Memory:** the $\approx 28\,\%$ reduction for RK4-LS vs RK4 Classic is consistent with the reduction from 4 to 2 intermediate vectors.
- **Parallel time:** the absence of gain of RK4-LS Par over RK4-LS is expected for pure Python – this does not invalidate the method.

8 Conclusion

8.1 Technical Summary

This project has enabled the implementation, benchmarking and analysis of five time-integration schemes in the context of spacecraft thermal simulation. The main conclusions are:

1. **Accuracy:** RK4 schemes (classic and LS) achieve fourth-order accuracy, approximately 3 orders of magnitude better than Euler and 2 orders of magnitude better than RK2, for the same time step.
2. **Memory:** the RK4-LS scheme reduces memory overhead by approximately 28 % compared to classical RK4 for large dimensions, going from 4 to 2 intermediate vectors.
3. **Computation time:** in pure Python, RK4-LS is $\approx 35\,\%$ slower than classical RK4 due to the 5 stages versus 4.
4. **Parallelism:** `numexpr` parallelism is only profitable beyond $\sim 5 \times 10^5$ components. Amdahl's Law theoretically limits the maximum speedup given the irreducible sequential fraction (temporal step dependency).
5. **Case study – Spacecraft thermal equation:** RK4-LS directly applies to the full nodal thermal balance (10), handling conductive and radiative exchanges between nodes, as formalised by the matrix system (11).

8.2 Perspectives

Future work could explore higher-order LSRK schemes (order 5 or 6) for models requiring increased accuracy, as well as GPU-based parallelisation of the vector operations to more fully exploit data parallelism for large-scale thermal models.

References

[1] M. H. Carpenter and C. A. Kennedy, *Fourth-Order 2N-Storage Runge-Kutta Schemes*, NASA Technical Memorandum 109112, NASA Langley Research Center, 1994.

[2] Wikipedia, "Amdahl's law," https://en.wikipedia.org/wiki/Amdahl%27s_law, accessed March 2026.

[3] Wikipedia, "Runge-Kutta methods," https://en.wikipedia.org/wiki/Runge%E2%80%93Kutta_methods, accessed March 2026.

[4] C. W. Gear, "Parallel Methods for Ordinary Differential Equations," *Invited paper at the International Symposium on Vector and Parallel Processors for Scientific Computation*, Rome, 1987.

[5] J. Nievergelt, "Parallel Methods for Integrating Ordinary Differential Equations," *Communications of the ACM*, vol. 7, no. 12, pp. 731–733, 1964.

[6] D. Cooke et al., *numexpr: Fast numerical expression evaluator for NumPy*, https://github.com/pydata/numexpr, 2023.

A Python Code Listings

A.1 RK4 Low Storage – Carpenter & Kennedy

```python
def rk4_ls(f, t, h, y):
    a = np.array([0., -567301805773./1357537059087.,
                  -2404267990393./2016746695238.,
                  -3550918686646./2091501179385.,
                  -1275806237668./842570457699.])
    b = np.array([1432997174477./9575080441755.,
                  5161836677717./13612068292357.,
                  1720146321549./2090206949498.,
                  3134564353537./4481467310338.,
                  2277821191437./14882151754819.])
    d = np.zeros(y.shape[1])    # Only one temporary vector
    for i in range(len(t) - 1):
        yi = y[i].copy()        # Current state
        d[:] = 0
        for s in range(5):
            d = a[s] * d + h * f(t[i], yi)
            yi = yi + b[s] * d
        y[i + 1] = yi
    return y
```

Listing 2: RK4 Low Storage implementation (Carpenter & Kennedy)

A.2 Spacecraft Nodal Thermal Model

```python
import numpy as np

def nodal_thermal_case_study(n=20, tf=5400.0, orbit_period=5400.0):
    sigma = 5.670374419e-8   # Stefan-Boltzmann constant (W/m^2/K^4)

    rng = np.random.default_rng(42)
    MC      = rng.uniform(500.0, 2000.0, n)
    area    = rng.uniform(0.01,  0.10,   n)
    alpha_s = rng.uniform(0.2,   0.9,    n)
    eps     = rng.uniform(0.5,   0.95,   n)
    Q_diss  = rng.uniform(0.0,   5.0,    n)

    GL = np.zeros((n, n))
    for i in range(n - 1):
        g = rng.uniform(0.1, 2.0)
        GL[i, i+1] = GL[i+1, i] = g

    GR = np.zeros((n, n))
    for i in range(n - 1):
        gr = rng.uniform(1e-4, 5e-3)
        GR[i, i+1] = GR[i+1, i] = gr

    phi_S = 1361.0
    phi_a = 0.30 * phi_S * 0.30
    phi_T = 237.0

    def sunlit_factor(t):
        return 1.0 if (t % orbit_period) / orbit_period < 0.6 else 0.0

    def f_thermal(t, T):
        sf = sunlit_factor(t)
        dTdt = np.zeros(n)
        for i in range(n):
            flux  = alpha_s[i] * area[i] * phi_S * sf
            flux += alpha_s[i] * area[i] * phi_a * sf
            flux += eps[i]     * area[i] * phi_T
            flux += Q_diss[i]
            flux += np.sum(GL[i, :] * (T - T[i]))
            flux += sigma * np.sum(GR[i, :] * (T**4 - T[i]**4))
            dTdt[i] = flux / MC[i]
        return dTdt

    GL_max = np.max(np.sum(GL, axis=1))
    MC_min = np.min(MC)
    h = 0.5 * 2.79 / (GL_max / MC_min)
    N = int(tf / h) + 1
    t = np.linspace(0.0, tf, N)
    h = t[1] - t[0]

    a_rk = [0., -567301805773./1357537059087.,
            -2404267990393./2016746695238.,
            -3550918686646./2091501179385.,
            -1275806237668./842570457699.]
    b_rk = [1432997174477./9575080441755.,
            5161836677717./13612068292357.,
            1720146321549./2090206949498.,
```

```
            3134564353537./4481467310338.,
            2277821191437./14882151754819.]

    T = np.full(n, 250.0)
    d = np.zeros(n)
    T_hist = [T.copy()]
    t_save = [0.0]
    save_every = max(1, N // 300)

    for i in range(N - 1):
        d[:] = 0.0
        for s in range(5):
            k = f_thermal(t[i], T)
            d = a_rk[s] * d + h * k
            T = T + b_rk[s] * d
        if i % save_every == 0:
            T_hist.append(T.copy())
            t_save.append(t[i + 1])

    return np.array(t_save), np.array(T_hist)

t_save, T_hist = nodal_thermal_case_study()
```

Listing 3: Spacecraft nodal thermal model with RK4-LS